Beyond the Basics

Pseudocode · Python · Flowcharts · Logic Gates

Explanations, worked examples and practice questions with full answers for everything **after** the basics (variables, input/output and decisions).

Based on the **Cambridge O Level Computer Science 2210** syllabus for 2026–2028, Paper 2: Algorithms, Programming and Logic (Topics 7–10), plus Python for the 15-mark scenario question.

Compiled 12 November 2026

Contents

How this guide works 4

1. Iteration and nested constructs 4

1.1 The three loop types . . . 4

1.2 Totalling and counting . . . 4

1.3 Nested statements . . . 5

1.4 Python translation . . . 5

1.5 Worked example — trace of a REPEAT loop . . . 5

2. String handling 7

2.1 The four string operations . . . 7

2.2 Processing strings character by character . . . 8

2.3 Python translation . . . 8

3. Arrays 9

3.1 One-dimensional arrays . . . 9

3.2 Two-dimensional arrays . . . 10

3.3 Python translation . . . 10

4. Procedures, functions and modularity 11

4.1 Why split up code? . . . 11

4.2 Procedures . . . 11

4.3 Functions . . . 12

4.4 Local and global variables . . . 12

5. File handling 13

5.1 Why files? . . . 13

5.2 The four file operations . . . 13

5.3 Typical pattern: process a file of values . . . 14

6. Standard algorithms, trace tables and debugging 15

6.1 The seven standard methods . . . 15

6.2 Linear search . . . 15

6.3 Bubble sort . . . 16

6.4 Trace tables and dry-runs . . . 16

6.5 Common logic errors to spot in error-finding questions . . . 16

7. Program development, testing and data checking 19

7.1 The program development life cycle (PDLC) . . . 19

7.2 Validation — is the data reasonable? . . . 19

7.3 Verification — was it entered accurately? . . . 19

7.4 Test data — normal, abnormal, extreme, boundary . . . 19

8. Flowcharts **20**

8.1 The six standard symbols . . . 20

8.2 Loop patterns . . . 21

8.3 Converting between pseudocode and flowcharts . . . 21

9. Boolean logic and logic gates **22**

9.1 The seven gates . . . 22

9.2 The three conversions . . . 23

10. Databases **25**

10.1 Key concepts . . . 25

10.2 SQL — the examinable keywords . . . 25

11. Python for the scenario question **26**

11.1 The rules . . . 26

11.2 Pseudocode → Python cheat table . . . 27

11.3 Strategy for a strong 15-mark answer . . . 27

11.4 Worked mini-scenario . . . 28

1 - collect 10 valid values **29**

2 - total and average **29**

3 - highest value **29**

4 - count days above average **29**

Appendix — command words **29**

How this guide works

This guide covers everything **after** the basics — you already know variables, input/output, operators and decisions (IF/CASE), so those are skipped. What remains is the heart of Paper 2: iteration, strings, arrays, subroutines, files, standard algorithms, testing, flowcharts, Boolean logic, databases, and Python for the scenario question.

Each section gives you tight explanations and worked examples, then practice questions with model answers and mark allocations. Try the questions **before** looking at the answers.

> **Note**
> **The golden exam rule.** In Paper 2, every coding answer must be written in **Cambridge pseudocode** — answers in Python (or any real language) score zero, **except** in the final 15-mark scenario question, where pseudocode **or** Python/Visual Basic/Java is allowed. Syntax is never marked: the **logic** is what earns marks. But operators must be precise: write `x > y`, never "x is greater than y".

The guide mirrors the syllabus pseudocode exactly (assignment arrow ←, keywords in upper case, `DECLARE / INPUT / OUTPUT / IF...ENDIF / WHILE...DO...ENDWHILE / REPEAT...UNTIL / FOR...NEXT / PROCEDURE / FUNCTION`). Python translations are shown in grey boxes where useful.

1. Iteration and nested constructs

1.1 The three loop types

Cambridge pseudocode has exactly three loop forms. Choosing the right one (and knowing when each runs **zero**, **one**, or **many** times) is a favourite exam target.

FOR — count-controlled. Use when you know in advance how many times to repeat.

```
FOR I ← 1 TO 10
    OUTPUT I
NEXT I
```

- The counter (`I` here) is assigned every integer value from the start to the end value **inclusive**.
- If start > end, the loop body never runs.
- A step size can be given — including negative steps:

```
FOR I ← 1 TO 9 STEP 2      // I takes 1, 3, 5, 7, 9
    OUTPUT I
NEXT I
```

```
FOR Countdown ← 10 TO 1 STEP -1
OUTPUT Countdown
NEXT Countdown
```

WHILE — pre-condition. Use when you repeat **while** something stays true and it might never be true at all. The test happens **before** each run, so the body can run zero times.

```
WHILE Password <> "Secret" DO
    INPUT Password
ENDWHILE
```

REPEAT ... UNTIL — post-condition. Use when the body must run **at least once**. The test happens **after** each run, and the loop **stops when the condition becomes TRUE** (this is the opposite of Python's `while`, a classic trap).

```
REPEAT
    INPUT Password
UNTIL Password = "Secret"       // stops when TRUE
```

1.2 Totalling and counting

Two patterns you will use constantly:

```
Total ← 0
FOR I ← 1 TO 5
    INPUT Value
    Total ← Total + Value
NEXT I
```

```
Count ← 0
FOR I ← 1 TO 5
    INPUT Value
    IF Value > 10
    THEN
        Count ← Count + 1
    ENDIF
NEXT I
```

Always initialise the accumulator (`Total ← 0`) or counter (`Count ← 0`) **before** the loop.

1.3 Nested statements

Loops and IF statements can be placed inside each other. The syllabus never requires more than **three levels of nesting**, and indentation must make the structure obvious (four spaces per level; THEN/ELSE two extra spaces). Worked nested example — one total per row of a 2D structure:

```
FOR Row ← 1 TO 3
    RowTotal ← 0
    FOR Column ← 1 TO 4
        RowTotal ← RowTotal + Grid[Row, Column]
    NEXT Column
    OUTPUT "Total for row ", Row, " is ", RowTotal
NEXT Row
```

1.4 Python translation

> **Note**
> In Python: `FOR I ← 1 TO 10` becomes `for i in range(1, 11):` (range stops **before** the end value). `REPEAT ... UNTIL cond` becomes `while True:` with `if cond: break` at the **end** of the body. `WHILE cond DO` becomes `while cond:`. Remember: pseudocode loops are 1-based and inclusive; Python range is 0-based and exclusive.

1.5 Worked example — trace of a REPEAT loop

Trace tables track every variable and output at each step. For inputs 5, −3, 2, 0:

```
Total ← 0
Count ← 0
REPEAT
    INPUT Value
    IF Value > 0
        THEN
            Total ← Total + Value
            Count ← Count + 1
    ENDIF
UNTIL Value = 0
OUTPUT Total, Count
```

Step	Value	Total	Count
before loop	-	0	0
1	5	5	1
2	-3	5	1
3	2	7	2
4	0	7	2

The loop stops after the input 0 (condition `Value = 0` now TRUE). Final output: `7, 2`.

Q1. [3 marks]

Write pseudocode for an algorithm that calculates the sum of the squares of the numbers 1 to 10 (that is 1 + 4 + 9 + ... + 100) and outputs the total. The value 385 should not appear as a literal in your code.

Answer 1

The total must be built up with an accumulator using the square of the loop counter.

```
Total ← 0
FOR I ← 1 TO 10
    Total ← Total + I ^ 2
NEXT I
OUTPUT Total
```

Mark points: initialising `Total` (1); correct loop 1 to 10 and accumulating `I ^ 2` (1); OUTPUT of the calculated total (1). Accept `I * I` instead of `I ^ 2`.

Q2. [4 marks]

The following program is run with inputs 5, −3, 2, 0 in that order. Complete a trace table showing `Value`, `Total` and `Count` at each step, and state what is output.

```
Total ← 0
Count ← 0
REPEAT
    INPUT Value
    IF Value > 0
        THEN
            Total ← Total + Value
            Count ← Count + 1
    ENDIF
UNTIL Value = 0
OUTPUT Total, Count
```

Answer 2

T:
Step	Value	Total	Count
before loop	-	0	0
1	5	5	1
2	-3	5	1
3	2	7	2
4	0	7	2
Output: 7, 2.
Mark points: `Total` column correct after each input (2); `Count` column correct (1); correct final output (1). Note that −3 is ignored (not > 0) and that 0 is processed before the loop stops, because `UNTIL` is tested after the body.

Q3. [3 marks]

Rewrite this FOR loop using a WHILE loop. It should produce exactly the same output.

```
FOR I ← 1 TO 10
    OUTPUT I * 2
NEXT I
```

Answer 3

```
I ← 1
WHILE I <= 10 DO
    OUTPUT I * 2
    I ← I + 1
ENDWHILE
```

Mark points: initialise `I ← 1` (1); correct condition `I <= 10` (1); increment `I ← I + 1` and correct WHILE structure (1). Common mistake: using < instead of <=, which prints only 8 values (2 to 16).

Q4. [2 marks]

This loop is supposed to output the numbers 1 to 5, but it only outputs 1 to 4. Identify the error and write the corrected line.

```
I ← 1
WHILE I < 5 DO
    OUTPUT I
    I ← I + 1
ENDWHILE
```

Answer 4

The condition `I < 5` should be `I <= 5`. Mark points: identifying the wrong condition (1); correcting it (1).

Q5. [2 marks]

Translate this pseudocode into Python. Your code should print one number.

```
Total ← 0
FOR I ← 1 TO 9 STEP 2
    Total ← Total + I
NEXT I
OUTPUT Total
```

Answer 5

```
total = 0
for i in range(1, 10, 2):
    total = total + i
print(total)
```

It prints 25 (1 + 3 + 5 + 7 + 9). Mark points: correct loop with `range(1, 10, 2)` (1); accumulation and print (1). Remember range stops before its end value.

2. String handling

2.1 The four string operations

Strings are examined heavily because they combine naturally with loops and selection.

Operation	Meaning	Example	Result
`LENGTH(s)`	number of characters	`LENGTH("Happy Days")`	`10`
`SUBSTRING(s, start, length)`	substring of given length from position start (normally 1 = first character)	`SUBSTRING("Happy Days", 1, 5)`	`"Happy"`

Operation	Meaning	Example	Result
`UCASE(s)`	all characters upper case	`UCASE("Happy")`	`"HAPPY"`
`LCASE(s)`	all characters lower case	`LCASE('W')`	`'w'`

Strings join with the + operator: `FullName ← FirstName + " " + LastName`. Positions count from 1 (Cambridge normally uses 1-based positions — questions will make this clear).

2.2 Processing strings character by character

The standard pattern is a FOR loop from 1 to `LENGTH(s)`, extracting one character at a time with `SUBSTRING(s, I, 1)`:

```
// count how many vowels are in S
Count ← 0
FOR I ← 1 TO LENGTH(S)
    Ch ← UCASE(SUBSTRING(S, I, 1))
    IF Ch = "A" OR Ch = "E" OR Ch = "I" OR Ch = "O" OR Ch = "U"
        THEN
            Count ← Count + 1
    ENDIF
NEXT I
OUTPUT Count
```

Common string tasks you should be able to write: counting occurrences of a character, reversing a string (loop downwards with `STEP -1`), checking a password against a rule, extracting parts of a formatted code.

2.3 Python translation

Note

Python equivalents: `LENGTH(s)` → `len(s)`; `UCASE(s)` → `s.upper()`; `LCASE(s)` → `s.lower()`; `SUBSTRING(s, start, length)` → `s[start-1 : start-1+length]`. Watch the offset: `SUBSTRING(S, 4, 3)` is `s[3:6]`. Python strings are 0-based; Cambridge pseudocode is normally 1-based.

Q1. [3 marks]

The variable `Text` contains "Computer". Evaluate:
(a) `LENGTH(Text)` (b) `SUBSTRING(Text, 3, 3)` (c) `UCASE(SUBSTRING(Text, 2, 1))`

Answer 1

(a) 8 (b) "mpu" (c) "O".
Mark points: one per part. Positions: C=1, o=2, m=3, p=4, u=5, t=6, e=7, r=8. `SUBSTRING(Text, 3, 3)` starts at "m" and takes 3 characters; `SUBSTRING(Text, 2, 1)` is "o", which UCASE turns into "O".

Q2. [4 marks]

Write pseudocode that inputs a string S and counts how many times the letter "e" appears, in either upper or lower case. Output the count.

Answer 2

```
INPUT S
Count ← 0
FOR I ← 1 TO LENGTH(S)
    Ch ← UCASE(SUBSTRING(S, I, 1))
    IF Ch = "E"
        THEN
            Count ← Count + 1
    ENDIF
NEXT I
OUTPUT Count
```

Mark points: loop over the whole string using LENGTH (1); extracting the character with SUBSTRING (1); case-insensitive comparison using UCASE or LCASE (1); counting and outputting (1). Accept comparison against both "e" and "E" instead of converting case.

Q3. [3 marks]

Write pseudocode that inputs a string S and outputs it reversed. For input "ABC" the output must be "CBA".

Answer 3

```
INPUT S
Reversed ← ""
FOR I ← LENGTH(S) TO 1 STEP -1
    Reversed ← Reversed + SUBSTRING(S, I, 1)
NEXT I
OUTPUT Reversed
```

Mark points: loop downwards from `LENGTH(S)` to 1 (1); `STEP -1` and character extraction (1); concatenation into `Reversed` and output (1).

Q4. [2 marks]

This Python code is run. Write down the exact output.

```
s = "Computer"
print(len(s), s[2:5], s.upper())
```

Answer 4

`8 mpu COMPUTER` (on one line). Mark points: 8 and `mpu` (1) — remember Python slices are 0-based, so `s[2:5]` is characters at indexes 2, 3, 4; `COMPUTER` (1).

3. Arrays

3.1 One-dimensional arrays

An array is a fixed-length block of elements of the **same** data type, accessed by index. Declare with explicit bounds (the lower bound is normally 1):

```
DECLARE StudentNames : ARRAY[1:30] OF STRING
DECLARE Marks : ARRAY[1:30] OF INTEGER
```

Read and write single elements with square brackets: `Marks[5] ← 72`, `OUTPUT StudentNames[3]`. An index may be a variable or expression: `Marks[I]`, `Marks[I + 1]`.

Arrays are nearly always used **with loops** — to fill them, print them, search them, or sort them:

```
FOR I ← 1 TO 30
```

```
    INPUT Marks[I]
NEXT I
```

3.2 Two-dimensional arrays

A 2D array is declared with two bounds and accessed with two indices separated by a comma:

```
DECLARE NoughtsAndCrosses : ARRAY[1:3, 1:3] OF CHAR
NoughtsAndCrosses[2, 3] ← 'X'
```

Typical operations need **nested loops**: every row × every column. To walk down a column instead, swap the loop order.

3.3 Python translation

Note

A Cambridge array of size 30 maps to a Python **list**. Python lists are dynamic and 0-based: `names = [""] * 30` then `names[i]` with `i` from 0. The pseudocode `Grid[2, 3]` corresponds to `grid[2][3]` (a list of lists). In the scenario question either style is fine — just stay consistent.

Q1. [3 marks]

Write pseudocode to declare an array capable of holding 20 integer exam marks, then fill it with values typed by the user.

Answer 1

```
DECLARE Marks : ARRAY[1:20] OF INTEGER
FOR I ← 1 TO 20
    INPUT Marks[I]
NEXT I
```

Mark points: declaration with type and bounds (1); loop from 1 to 20 (1); INPUT into the array element using the loop counter as index (1).

Q2. [4 marks]

The array A contains the values 2, 7, 4, 9 in positions 1 to 4. Trace the algorithm below, showing `I`, `A[I]` and `Max` at each step of the loop, and state what is output.

```
Max ← A[1]
FOR I ← 2 TO 4
    IF A[I] > Max
        THEN
            Max ← A[I]
    ENDIF
NEXT I
OUTPUT Max
```

Answer 2

T:
Step	I	A[I]	Max
before loop	-	-	2
1	2	7	7
2	3	4	7
3	4	9	9
Output: 9.
Mark points: `Max` starts at 2 (1); each correct row of the trace (3). Award the rows mark by mark: 7 replaces 2, 4 is ignored, 9 replaces 7.

Q3. [4 marks]

A 2D array `Grid` has 2 rows and 3 columns. Row 1 contains 3, 1, 4 and row 2 contains 2, 5, 6. Write pseudocode to output the total of each row (your program must work for any values stored in the array).

Answer 3

```
FOR Row ← 1 TO 2
    Sum ← 0
    FOR Col ← 1 TO 3
        Sum ← Sum + Grid[Row, Col]
    NEXT Col
    OUTPUT "Row ", Row, " total is ", Sum
NEXT Row
```

This outputs `Row 1 total is 8` then `Row 2 total is 13`.
Mark points: nested loop covering rows and columns (2); accumulation with both indices (1); correct totals output inside the outer loop (1).

Q4. [2 marks]

Translate the pattern below into Python: build a list called `squares` holding the squares of the numbers 1 to 5 (1, 4, 9, 16, 25) by looping and appending.

Answer 4

```
squares = []
for i in range(1, 6):
    squares.append(i * i)
```

Mark points: empty list plus `.append(...)` (1); loop with `range(1, 6)` producing 1 to 5 (1). Accept `squares = [i * i for i in range(1, 6)]`.

4. Procedures, functions and modularity

4.1 Why split up code?

Topic 7 expects you to understand **decomposition** (breaking a problem into sub-systems) and Topic 8 expects **maintainable programs**: meaningful identifiers, comments (`//`), and subroutines. A program built from small procedures and functions is easier to write, test, read and reuse.

4.2 Procedures

A procedure performs a task and returns nothing. It is defined at the start of the code and called with `CALL`:

```
PROCEDURE PrintLine(Symbol : CHAR, Times : INTEGER)
    DECLARE I : INTEGER
    DECLARE LineText : STRING
    LineText ← ""
    FOR I ← 1 TO Times
        LineText ← LineText + Symbol
    NEXT I
    OUTPUT LineText
ENDPROCEDURE
```

CALL PrintLine('*', 10)

- Parameters are typed (`Symbol : CHAR`); up to **three** parameters are allowed.

- The call `CALL PrintLine('*', 10)` is a complete statement.

4.3 Functions

A function performs a task and **returns one value**, so it is used inside expressions — never with `CALL`:

```
FUNCTION Max(Value1 : INTEGER, Value2 : INTEGER) RETURNS INTEGER
    IF Value1 > Value2
        THEN
            RETURN Value1
        ELSE
            RETURN Value2
    ENDIF
ENDFUNCTION
```

OUTPUT "The winner is ", Max(Score1, Score2)

The definition states the return type (`RETURNS INTEGER`) and `RETURN` sends the value back to the point of call.

4.4 Local and global variables

A variable declared **inside** a procedure or function is **local** — it exists only there, and shadows any global of the same name. A variable declared at the top of the program is **global** — visible everywhere. Parameters behave like locals.

Q1. [3 marks]

Write a procedure `PrintLine` with parameters `Symbol : CHAR` and `Times : INTEGER` that outputs a single line consisting of `Times` copies of `Symbol`. Then call it to print 10 stars.

Answer 1

```
PROCEDURE PrintLine(Symbol : CHAR, Times : INTEGER)
    DECLARE I : INTEGER
    DECLARE LineText : STRING
    LineText ← ""
    FOR I ← 1 TO Times
        LineText ← LineText + Symbol
    NEXT I
    OUTPUT LineText
ENDPROCEDURE
```

CALL PrintLine('*', 10)

```
Mark points: correct procedure header with both typed parameters (1); loop from 1 to __Times__ building the str
```

Q2. [4 marks]

Write a function `IsEven(Number : INTEGER) RETURNS BOOLEAN` that returns TRUE when `Number` is even. Then use it in a program that inputs 10 integers and outputs how many of them are even.

Answer 2

```
FUNCTION IsEven(Number : INTEGER) RETURNS BOOLEAN
    RETURN MOD(Number, 2) = 0
ENDFUNCTION
```

DECLARE I : INTEGER
DECLARE Value : INTEGER
DECLARE Count : INTEGER
Count ← 0
FOR I ← 1 TO 10

```
INPUT Value
IF IsEven(Value)
THEN
Count ← Count + 1
ENDIF
NEXT I
OUTPUT Count
```

```
Mark points: function header with __RETURNS BOOLEAN__ (1); __MOD(Number, 2) = 0__ test (1); loop taking 10 inpu
```

Q3. [3 marks]

This program uses a function. State exactly what it outputs, and show your working.

```
FUNCTION Square(X : INTEGER) RETURNS INTEGER
    RETURN X * X
ENDFUNCTION
```

```
DECLARE N : INTEGER
N ← 3
OUTPUT Square(N) + Square(N + 1)
```

Answer 3

25. Working: `Square(3)` = 3 × 3 = 9; `Square(4)` = 4 × 4 = 16; 9 + 16 = 25.
Mark points: 9 (1); 16 (1); 25 (1). Function calls are expressions — the arguments are evaluated, the returned values replace the calls, then the addition happens.

Q4. [2 marks]

What does this program output? Explain why.

```
DECLARE Total : INTEGER
Total ← 5
PROCEDURE AddUp
    DECLARE Total : INTEGER
    Total ← 10
ENDPROCEDURE
CALL AddUp
OUTPUT Total
```

Answer 4

It outputs 5. The `Total` declared inside the procedure is a **local** variable that shadows the global one; changing it to 10 leaves the global `Total` (5) untouched.
Mark points: correct output (1); explanation in terms of local vs global variables (1).

5. File handling

5.1 Why files?

Variables and arrays vanish when the program ends. A **file** on backing storage keeps data between program runs — for long-term storage, large data sets, or moving data between programs.

5.2 The four file operations

```
OPENFILE "Marks.txt" FOR READ      // or FOR WRITE
READFILE "Marks.txt", Mark
WRITEFILE "Marks.txt", Mark
CLOSEFILE "Marks.txt"
```

- Open a file in **one mode at a time**.
- `FOR READ` reads existing data; `FOR WRITE` creates a new file and **destroys anything the file already contained**.
- `READFILE` / `WRITEFILE` move one item (or one line of text) at a time, into/out of a variable.
- Always `CLOSEFILE` when finished.

5.3 Typical pattern: process a file of values

```
OPENFILE "Scores.txt" FOR READ
Total ← 0
FOR I ← 1 TO 10
    READFILE "Scores.txt", Mark
    Total ← Total + Mark
NEXT I
CLOSEFILE "Scores.txt"
OUTPUT "Average is ", Total / 10
```

Note

The syllabus does not define an "end of file" test, so exam questions say how many items the file holds, or end the data with a sentinel value such as −1. Check the question for which pattern to use.

Q1. [5 marks]

The text file "Scores.txt" contains exactly 10 integer exam marks, one per line. Write pseudocode to output the average (mean) of these marks.

Answer 1

```
OPENFILE "Scores.txt" FOR READ
Total ← 0
FOR I ← 1 TO 10
    READFILE "Scores.txt", Mark
    Total ← Total + Mark
NEXT I
CLOSEFILE "Scores.txt"
OUTPUT "Average is ", Total / 10
```

Mark points: `OPENFILE ... FOR READ` (1); loop 10 times (1); `READFILE` into a variable and accumulate (1); divide by 10 and output (1); `CLOSEFILE` (1).

Q2. [2 marks]

State what happens to the existing contents of "Notes.txt" when this statement runs, and why.

```
OPENFILE "Notes.txt" FOR WRITE
```

Answer 2

The existing contents are **erased/lost** — opening `FOR WRITE` creates a new empty file (it does not append). Anything previously stored in "Notes.txt" is overwritten.
Mark points: data is lost/overwritten (1); because `WRITE` mode creates a new file (does not add to existing data) (1).

Q3. [3 marks]

A student wants to copy the contents of "Old.txt" into "New.txt". Their code opens "Old.txt" and then writes to it with `WRITEFILE`. Explain what is wrong and give the corrected opening lines.

Answer 3

A file opened `FOR READ` cannot be written to with `WRITEFILE`; also, writing must go to the **destination** file. "Old.txt" must be opened `FOR READ` and "New.txt" `FOR WRITE` (two files open at once, one per mode):

```
OPENFILE "Old.txt" FOR READ
OPENFILE "New.txt" FOR WRITE
```

Mark points: mode mismatch — cannot `WRITEFILE` to a file opened for reading (2); correct separate opens (1).

Q4. [2 marks]

Write the Python equivalent of the pseudocode in Question 1 (read 10 marks from "Scores.txt" and print their average).

Answer 4

```
total = 0
f = open("Scores.txt", "r")
for i in range(10):
    mark = int(f.readline())
    total = total + mark
f.close()
print(total / 10)
```

Mark points: opening and reading inside a loop of 10 (1); closing the file and printing the average (1). Accept `lines = f.readlines()` then a loop, or `map(int, ...)`, provided exactly 10 values are used.

6. Standard algorithms, trace tables and debugging

6.1 The seven standard methods

The syllabus limits "standard methods of solution" to exactly these — you must be able to **write**, **trace** and **apply** each one:

- **Linear search** — find a value in an array
- **Bubble sort** — sort an array into order
- **Totalling** — add up values (Section 1)
- **Counting** — count values matching a condition (Section 1)
- **Finding the maximum** — largest value
- **Finding the minimum** — smallest value
- **Finding the average** — total ÷ count

Maximum (and similarly minimum):

```
Max ← A[1]
FOR I ← 2 TO 12
    IF A[I] > Max
        THEN
            Max ← A[I]
    ENDIF
NEXT I
```

6.2 Linear search

Walk through the array comparing each element to the search key; stop when found:

```
Found ← FALSE
```

```
Position ← 0
I ← 1
WHILE Found = FALSE AND I <= 4 DO
    IF A[I] = SearchKey
        THEN
            Found ← TRUE
            Position ← I
    ENDIF
    I ← I + 1
ENDWHILE
IF Found = TRUE
    THEN
        OUTPUT "Found at position ", Position
    ELSE
        OUTPUT "Not found"
ENDIF
```

6.3 Bubble sort

Repeatedly sweep through the array, swapping neighbouring pairs that are out of order. Each sweep "bubbles" the next largest value to its correct place:

```
FOR Outer ← 1 TO N - 1
    FOR Inner ← 1 TO N - Outer
        IF A[Inner] > A[Inner + 1]
            THEN
                Temp ← A[Inner]
                A[Inner] ← A[Inner + 1]
                A[Inner + 1] ← Temp
        ENDIF
    NEXT Inner
NEXT Outer
```

Worked trace sorting 5, 3, 4 (ascending):

Pass	Comparison	Swap?	Array after step
1	5 vs 3	yes	3, 5, 4
1	5 vs 4	yes	3, 4, 5
2	3 vs 4	no	3, 4, 5
2	4 vs 5	no	3, 4, 5

Two "no swap" comparisons in a row mean the array is sorted.

6.4 Trace tables and dry-runs

A trace table documents a dry-run of an algorithm. Give **one column per variable** (plus one for outputs/prompts) and **one row per step** — usually one row per loop iteration. Exam tips:

- Write down the initial values row first.
- Show the row **after** each iteration's changes.
- If asked for outputs, add an OUTPUT column and write each output when it happens.
- Keep units/types consistent (e.g. don't put "TRUE" in a number column).

6.5 Common logic errors to spot in error-finding questions

- **Missing increment** → infinite loop (the loop counter never changes).
- **Wrong comparison** → < instead of <= (off-by-one: one value too few).
- **Uninitialised accumulator/counter** → garbage starting total.
- **Array index out of range** → reading `A[N + 1]` in a bubble sort whose inner loop goes to `N`.
- **Wrong operator** → = used for assignment (should be ←), words used instead of operator symbols.

Q1. [4 marks]

The array A holds 7, 2, 9, 4 in positions 1 to 4 and SearchKey is 9. Complete a trace table for the linear search below (columns I, A[I], Found, Position) and state the output.

```
Found ← FALSE
Position ← 0
I ← 1
WHILE Found = FALSE AND I <= 4 DO
    IF A[I] = SearchKey
        THEN
            Found ← TRUE
            Position ← I
    ENDIF
    I ← I + 1
ENDWHILE
OUTPUT Position
```

Answer 1

T:
Iteration	I	A[I]	Found	Position
before loop	1	-	FALSE	0
1	1	7	FALSE	0
2	2	2	FALSE	0
3	3	9	TRUE	3
Output: 3. The loop stops once Found becomes TRUE (search key 9 is at position 3), so iteration 4 never runs.
Mark points: three correct iterations (3); correct output 3 (1).

Q2. [5 marks]

Trace one complete bubble sort of the array `5, 3, 4` (ascending order). Show the array after every comparison and swap, and say how you know the sort is finished.

Answer 2

T:
Pass	Comparison	Action	Array after step
1	5 vs 3	swap	3, 5, 4
1	5 vs 4	swap	3, 4, 5
2	3 vs 4	no swap	3, 4, 5
2	4 vs 5	no swap	3, 4, 5
The sort is finished because pass 2 performs no swaps at all — every pair is already in order. Final array: `3, 4, 5`.
Mark points: first swap correct (2); second swap correct (1); pass 2 with no swaps (1); final array / reasoning (1).

Q3. [5 marks]

Write pseudocode for an algorithm that inputs 12 integers and outputs the **largest** and the **smallest** of them.

Answer 3

```
INPUT Value
Largest ← Value
Smallest ← Value
FOR I ← 2 TO 12
    INPUT Value
    IF Value > Largest
        THEN
            Largest ← Value
    ENDIF
    IF Value < Smallest
        THEN
            Smallest ← Value
    ENDIF
NEXT I
OUTPUT "Largest is ", Largest
OUTPUT "Smallest is ", Smallest
```

Mark points: initialising both with the first value (1); loop covering the remaining 11 values (1); update largest when bigger (1); update smallest when smaller (1); both outputs (1). Accept any clear alternative (e.g. an array filled first).

Q4. **[3 marks]**

This bubble sort should sort `A[1]` to `A[N]`, but it sometimes crashes. Identify the error and write the corrected inner loop line.

```
FOR Outer ← 1 TO N - 1
    FOR Inner ← 1 TO N
        IF A[Inner] > A[Inner + 1]
            THEN
                Temp ← A[Inner]
                A[Inner] ← A[Inner + 1]
                A[Inner + 1] ← Temp
        ENDIF
    NEXT Inner
NEXT Outer
```

Answer 4

When `Inner = N` the code reads `A[N + 1]`, which is **outside the array** (index out of range). The inner loop must stop one element early:

```
    FOR Inner ← 1 TO N - Outer
```

(`TO N - 1` is also acceptable.)
Mark points: identifying the out-of-range read of `A[N + 1]` (2); correcting the loop bound (1).

Q5. **[2 marks]**

Explain why programmers use dry-runs and trace tables before running a program.

Answer 5

They verify the **logic** of an algorithm step by step without executing it — variable values and outputs can be checked at each stage, which reveals errors (wrong conditions, uninitialised variables, bad arithmetic) early, before coding or when no computer is available.
Mark points: checking logic / finding errors before running (1); detail about tracking values at each step (1). Any two sensible benefits score the marks.

7. Program development, testing and data checking

7.1 The program development life cycle (PDLC)

Four stages — know what happens in **each** and which tasks belong where:

Stage	What happens
Analysis	Identify the problem and requirements; abstraction (focus on essentials); decomposition into parts
Design	Decomposition into sub-systems; structure diagrams; flowcharts; pseudocode
Coding	Write the program code; test iteratively as you go
Testing	Run the finished code against test data; find and fix remaining errors

7.2 Validation — is the data reasonable?

Validation checks run **automatically on input** to reject impossible or suspicious values before processing. The six check types in the syllabus:

- **Range check** — is a numeric value within limits? (e.g. age between 13 and 19)
- **Length check** — does a value have the right number of characters? (e.g. 6-character username)
- **Type check** — is it the right kind of data? (e.g. a number where a number is expected)
- **Presence check** — has anything been entered at all?
- **Format check** — does it match the expected pattern? (e.g. letters only, or digits in the right places)
- **Check digit** — an extra digit calculated from the others (e.g. barcodes, ISBNs); recalculated on receipt and compared to detect mistyping

7.3 Verification — was it entered accurately?

Verification checks that data was **transferred/recorded correctly** (as opposed to being reasonable):

- **Visual check** — a person re-reads the entered data against the source.
- **Double entry check** — the same data is entered twice and the two entries compared; a mismatch is flagged.

7.4 Test data — normal, abnormal, extreme, boundary

Testing needs data chosen to prove the program handles every case:

- **Normal** — typical valid value that should be accepted (e.g. 15).
- **Abnormal** — invalid data that must be rejected (e.g. "abc" typed for a number, or −5 for an age).
- **Extreme** — the **largest and smallest acceptable** values (13 and 19).
- **Boundary** — the extreme values **plus** the corresponding values that must be rejected: just outside on each side (12 and 20).

> **Note**
> Boundary data is where programs most often fail (off-by-one errors — < instead of <=). A good test plan always includes boundary values for every range.

Q1. [3 marks]

A registration program requires a username that is **exactly 6 characters long** and contains **letters only**. Identify the validation checks needed and say what each one checks.

Answer 1

- **Presence check** — that a username has been entered at all.
- **Length check** — that the username is exactly 6 characters.
- **Format check** — that it contains letters only (no digits or symbols).

Mark points: one per check, with a correct description of what it checks.

Q2. [4 marks]

An online competition form accepts only ages between **13 and 19 inclusive**. Suggest suitable test data of each type: (a) normal (b) abnormal (c) extreme (d) boundary.

Answer 2

(a) Normal: 15 (any valid age such as 13–19).
(b) Abnormal: "abc" (a word where a number is expected). Accept any clearly invalid value, e.g. −5 or 25.
(c) Extreme: 13 and 19 — the smallest and largest **acceptable** values.
(d) Boundary: 12, 13, 19, 20 — the extreme values plus the smallest and largest **rejected** values.
Mark points: one per category, with extreme and boundary giving values on **both** sides of the range.

Q3. [3 marks]

Classify each task by PDLC stage (analysis, design, coding or testing):
(a) Identifying what data the program must store (b) Drawing a structure diagram (c) Running the program with sample data to find errors

Answer 3

(a) Analysis (b) Design (c) Testing.
Mark points: one per task.

Q4. [2 marks]

Define **validation** and **verification**, making the difference between them clear.

Answer 4

Validation checks that input data is **reasonable/acceptable** (within range, right type/format) before it is processed. **Verification** checks that data has been **transferred or entered accurately** (e.g. by visual check or double entry). One checks sense; the other checks accuracy of entry.
Mark points: validation definition (1); verification definition (1).

8. Flowcharts

8.1 The six standard symbols

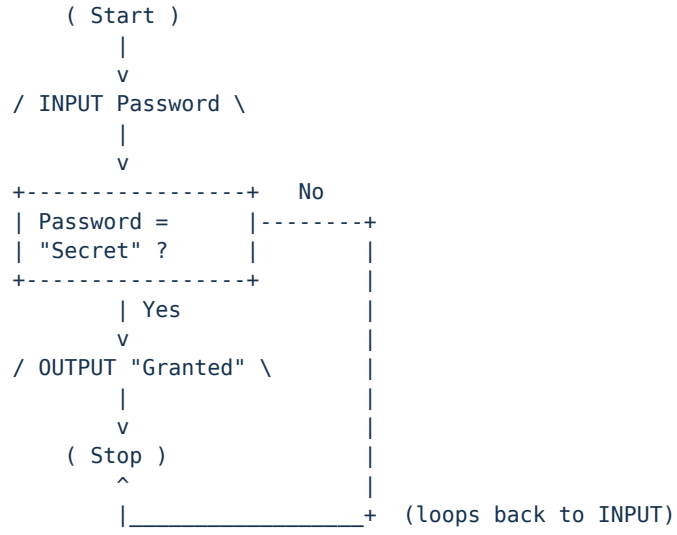
You must draw flowcharts with **exactly** these shapes — examiners penalise substitutes:

Symbol	Shape	Meaning
Terminator	Rounded box (stadium)	Start and Stop of the process
Process	Rectangle	Something done, e.g. Total ← Total + Value
Decision	Diamond	A Yes/No (True/False) question with exactly two exit lines
Input/Output	Parallelogram	INPUT or OUTPUT of data
Subroutine	Rectangle with double side bars	Call to a separate, non-linked flowchart
Flow line	Arrow	Control passing between shapes

8.2 Loop patterns

There is no "loop box" — loops are drawn with a **decision** and a **flow line looping back** to an earlier point. A WHILE-style loop tests before the body; a REPEAT-style loop tests after (its decision usually sits at the bottom).

Worked example — the password loop as an ASCII sketch (real answers are drawn with the shapes above):



8.3 Converting between pseudocode and flowcharts

- One box per action; keep assignments (←) inside **Process** boxes.
- Conditions (=, <, >, …) go in **Decision** diamonds; label both exits (Yes/No or True/False).
- Initialise counters/accumulators **before** the loop in their own Process box.
- A FOR loop becomes: initialiser → decision (counter within range?) → body → increment → flow line back to the decision.

Q1. [4 marks]

Draw a flowchart for this process: *Start. Repeatedly ask for a password until the user enters "Secret". Then output "Access granted" and stop.* (Describe each box, its shape, and where every flow line goes.)

Answer 1

- **Terminator** (rounded): "Start".
- **Input/Output** (parallelogram): "INPUT Password".
- **Decision** (diamond): "Password = 'Secret'?" — exit lines labelled **No** and **Yes**.
- The **No** line loops back to the flow line above the Input/Output box.
- The **Yes** line goes to an **Input/Output** (parallelogram): "OUTPUT 'Access granted'".
- **Terminator** (rounded): "Stop".

Mark points: correct shapes for terminator, input/output and decision (2); No branch looping back to the input (1); Yes branch with output and stop, labels present (1).

Q2. **[4 marks]**

Describe the flowchart for this pseudocode, naming each box's shape and its contents, and saying where the loop line goes.

```
Total ← 0
FOR I ← 1 TO 3
    INPUT Value
    Total ← Total + Value
NEXT I
OUTPUT Total
```

Answer 2

- **Terminator**: "Start".
- **Process** box: "Total ← 0", "I ← 1".
- **Decision** diamond: "I <= 3?" with a **Yes** exit going down and a **No** exit going to the right/down towards the output.
- Yes path: **Input/Output** "INPUT Value" → **Process** "Total ← Total + Value" → **Process** "I ← I + 1" → flow line back up to the **Decision**.
- No path: **Input/Output** "OUTPUT Total" → **Terminator** "Stop".

Mark points: initialisation in a process box (1); decision with `I <= 3` and a loop-back line (1); body boxes in the correct order (1); No path with output and stop (1).

Q3. **[2 marks]**

Sam drew a flowchart in which the decision was drawn as a parallelogram, and one exit of the decision had no label. State what is wrong in each case and how to fix it.

Answer 3

(a) A decision must be drawn as a **diamond** — a parallelogram is the Input/Output shape. (b) Both exit lines of a decision must be labelled (Yes/No or True/False) so the reader knows which condition leads where.

Mark points: one per error with the correction.

9. Boolean logic and logic gates

9.1 The seven gates

Logic values: TRUE = 1, FALSE = 0. The syllabus examines **NOT, AND, OR, NAND, NOR, XOR (also called EOR)**. NOT has a **single** input; all others have exactly **two**. Circuits are limited to **three inputs and one output**, drawn **exactly as the statement describes — never simplified**.

A	B	AND	OR	NAND	NOR	XOR
0	0	0	0	1	1	0

A	B	AND	OR	NAND	NOR	XOR
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	1	0	0	0

- **NOT** inverts its input: 0 → 1, 1 → 0.
- **NAND** = NOT AND: output 0 only when both inputs are 1.
- **NOR** = NOT OR: output 1 only when both inputs are 0.
- **XOR** outputs 1 only when the inputs are **different**.

9.2 The three conversions

You must convert freely in every direction between **problem statement**, **logic expression**, **truth table** and **logic circuit**. Worked example: *A greenhouse fan (X) runs when the temperature is high (A) AND the window is open (B), OR whenever the manual switch (C) is on.*

Expression: `X = (A AND B) OR C`

Circuit: an **AND** gate with inputs A and B; its output and input C feed an **OR** gate; the OR output is X.

A	B	C	A AND B	X = (A AND B) OR C
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

> **Note**
> Write expressions with explicit parentheses: `(A AND NOT B) OR C`. Pseudocode has no operator precedence rule for mixed logical operators, and examiners want the logic to be unambiguous. A 3-input truth table always has **8 rows** (000 to 111).

Q1. **[3 marks]**

Complete the truth table for `X = (A OR B) AND NOT C`.

Answer 1

T:
A	B	C	A OR B	NOT C	X
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	0	0
Output column: `0, 0, 1, 0, 1, 0, 1, 0`.
Mark points: correct intermediate columns (1); correct X column (2). Accept X column alone for 3 marks.

Q2. **[3 marks]**

Write a logic expression (and name the gate) for: *X is TRUE only when exactly one of the inputs A and B is TRUE.*

Answer 2

`X = (A AND NOT B) OR (B AND NOT A)`. This is the **XOR** gate, so `X = A XOR B` is also correct.
Mark points: correct expanded expression with AND, NOT and OR (2); correct parentheses (1). Full marks for `X = A XOR B` (or `A EOR B`).

Q3. **[3 marks]**

Draw the logic circuit for `X = A AND NOT B`. (Describe the gates and connections.)

Answer 3

A **NOT** gate with input B. An **AND** gate whose two inputs are A (direct) and the output of the NOT gate. The AND gate's output is X.
Mark points: identifying the AND gate (1); identifying the NOT gate on B (1); correct connections (1).

Q4. **[4 marks]**

Complete the truth table for `X = (A AND B) OR (NOT C)`.

Answer 4

T:
A	B	C	A AND B	NOT C	X
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1
Output column: `1, 0, 1, 0, 1, 0, 1, 1`.
Mark points: one mark per pair of correct rows (4).

Q5. **[2 marks]**

Compare the behaviour of the OR and XOR gates.

Answer 5

OR outputs 1 when **one or both** inputs are 1 (so 1 OR 1 = 1). **XOR** outputs 1 only when the inputs are **different** (so 1 XOR 1 = 0). They differ only in the case where both inputs are 1.
Mark points: OR behaviour (1); XOR behaviour including the both-1 case (1).

10. Databases

10.1 Key concepts

- A **table** holds **records** (rows) and **fields** (columns).
- Field data types: **text/alphanumeric**, **character**, **Boolean**, **integer**, **real**, **date/time**.
- A **primary key** is a field (or combination) that **uniquely identifies each record** — no duplicates, no empty values, never changes (e.g. `StudentID`).

10.2 SQL — the examinable keywords

SELECT · FROM · WHERE · ORDER BY ASCENDING · ORDER BY DESCENDING · SUM · COUNT · AND · OR

All questions use a **single table**. Worked example — table `Students`:

StudentID	Name	Class	Mark
101	Aisha	9A	84
102	Ben	9B	72
103	Chen	9A	91
104	Dina	9B	65
105	Elias	9A	78

```
SELECT Name FROM Students WHERE Class = "9A" ORDER BY Mark DESCENDING
```

Output: Chen (91), Aisha (84), Elias (78) — only 9A rows, highest mark first.

```
SELECT COUNT(StudentID) FROM Students WHERE Mark >= 80 AND Class = "9A"
```

Output: 2 (Aisha and Chen).

```
SELECT SUM(Mark) FROM Students WHERE Class = "9B"
```

Output: 137 (72 + 65).

Q1. [3 marks]

Write an SQL query to list the Name of every student in class "9A" with a Mark of 80 or more, showing the highest mark first.

Answer 1

```
SELECT Name FROM Students WHERE Class = "9A" AND Mark >= 80 ORDER BY Mark DESCENDING
```

Mark points: SELECT Name FROM Students (1); WHERE combining both conditions with AND (1); ORDER BY Mark DESCENDING (1).

Q2. [3 marks]

Using the Students table above, state the exact output of:

```
SELECT COUNT(StudentID) FROM Students WHERE Mark >= 70 AND Class = "9A"
```

Answer 2

3. The records matching **both** conditions are Aisha (84), Chen (91) and Elias (78) — each is in 9A **and** has a mark of at least 70. Ben has 72 but is in 9B, and Dina's 65 is below 70.
Mark points: correct output 3 (2); correctly identifying the matching records (1).

Q3. [2 marks]

A school table has the fields Name, DateOfBirth, Class and StudentID. Suggest a suitable primary key and justify your choice.

Answer 3

StudentID — it is unique to every student (unlike Name or Class), always has a value, and never changes.
Mark points: correct field (1); justification in terms of uniqueness/stability (1).

11. Python for the scenario question

11.1 The rules

The **last question of Paper 2** is a 15-mark unseen scenario (~30 minutes of your exam). You may answer in **pseudocode, Python, Visual Basic or Java** — every other question on the paper must be pseudocode. If you choose Python, the logic still earns the marks (good structure, correctness, completeness), not Python-specific cleverness. Write comments and meaningful variable names — the marking rewards **maintainable** code.

11.2 Pseudocode → Python cheat table

Cambridge pseudocode	Python
`DECLARE X : INTEGER`	`x = 0 (just name it; add a comment with the type)`
`CONSTANT Rate ← 5`	`RATE = 5`
`X ← X + 1`	`x = x + 1`
`INPUT X`	`x = int(input("..."))`
`OUTPUT A, "is", B`	`print(A, "is", B)`
`DIV(A, B)`	`A // B`
`MOD(A, B)`	`A % B`
`A ^ B`	`A ** B`
`X <> Y`	`X != Y`
`IF c THEN ... ELSE ... ENDIF`	`if c: ... else: ...`
`CASE OF X ... ENDCASE`	`if / elif / else`
`FOR I ← 1 TO N ... NEXT I`	`for i in range(1, N + 1):`
`FOR I ← A TO B STEP S`	`for i in range(A, B + 1, S):`
`WHILE c DO ... ENDWHILE`	`while c:`
`REPEAT ... UNTIL c`	`while True: ... if c: break`
`ARRAY[1:N] OF INTEGER`	`a = [0] * N (or just use append)`
`Grid[2, 3]`	`grid[2][3]`
`LENGTH(s)`	`len(s)`
`SUBSTRING(s, p, l)`	`s[p-1 : p-1+l]`
`UCASE(s) / LCASE(s)`	`s.upper() / s.lower()`
`ROUND(x, n) / RANDOM()`	`round(x, n) / random.random()`
`PROCEDURE P(x : INTEGER) ... CALL P(5)`	`def p(x): ... p(5)`
`FUNCTION F(x) RETURNS INTEGER ...`	`def f(x): return ...`
`OPENFILE "F" FOR READ`	`f = open("F", "r")`
`READFILE "F", v`	`v = f.readline()`
`WRITEFILE "F", v`	`f.write(str(v))`
`CLOSEFILE "F"`	`f.close()`

11.3 Strategy for a strong 15-mark answer

- **Read the scenario twice.** Underline the inputs, outputs, processing and any validation rules — this is the analysis stage and it prevents losing easy marks.
- **Plan on the question paper**: name your variables/arrays, sketch the main loop, decide whether to use a function or two (e.g. a `validate` function, a `find_highest` function).
- **Cover every requirement**: each bullet of the question maps to marks. A missing feature is missing marks.
- **Validate inputs** where the scenario implies limits, and handle the "not found"/empty case where relevant.
- **Use meaningful identifiers and comments** (the syllabus explicitly assesses maintainability).
- **Sanity-check with test data** mentally (a normal value, a boundary value) before moving on.

11.4 Worked mini-scenario

Ask the user for 5 daily temperatures. Store them. Output the average and the number of days above 25 degrees.

```
# Store 5 daily temperatures and report the average and hot days
temps = []
```

for day in range(1, 6): # 5 days
value = float(input("Temperature for day " + str(day) + ": "))
temps.append(value)

total = 0
for value in temps:
total = total + value
average = total / 5
print("Average temperature:", average)

hot_days = 0
for value in temps:
if value > 25:
hot_days = hot_days + 1
print("Days above 25 degrees:", hot_days)

Q1. **[10 marks]**

Scenario practice. The school canteen records how many meals were sold each day for 10 days. Write a complete Python program that:

- asks the user to enter the number of meals sold for each of the 10 days, rejecting and re-asking on any **negative** value;
- stores the 10 values;
- outputs the **total** and the **average** number of meals sold;
- outputs the **highest** number of meals sold;
- outputs **how many days** were above the average.

Answer 1

Model answer (other clear structures are equally acceptable):

```
def get_meals(day):
    """Ask for one day's sales, rejecting negative values."""
    while True:
        value = int(input("Meals sold on day " + str(day) + ": "))
        if value >= 0:
            return value
        print("Invalid: the number must not be negative")
```

def find_highest(values):
"""Return the largest value in the list."""
highest = values[0]
for v in values:
if v > highest:
highest = v
return highest

1 - collect 10 valid values

2 - total and average

3 - highest value

4 - count days above average

Appendix — command words

Knowing what each command word demands is free marks:

Command word	What it means
Calculate	work out from given facts, figures or information
Compare	identify similarities and/or differences
Define	give a precise meaning
Demonstrate	show how, or give an example
Describe	state the points / give characteristics and main features
Evaluate	judge quality, importance, amount or value
Explain	give purposes or reasons; make relationships clear; say why/how
Give	produce an answer from a source or from memory
Identify	name, select or recognise
Outline	set out the main points
Show (that)	provide structured evidence leading to a given result
State	express in clear terms
Suggest	apply knowledge to a situation with a range of valid responses